

Logic Programming languages.
Preliminaries on transition systems

Logic Programming Languages

The language

We will introduce a (family of) programming language(s) based on FOL, where programs are formulas (more precisely, clauses). A query for a program (or goal) is also a formula (clause).

Semantics

Since programs and queries are (set of) formulas, one can consider them as a FOL theory and provide a logical reading, exactly as we did for FOL:

- Model theory (also called declarative semantics in LP)
- Proof theory in terms of resolution (also called operational semantics in LP)
- Soundness and Completeness results

Logic Programming Languages

Computation as deduction

What is new for logic programming languages is that computation is exactly the same as logic deduction (resolution). This allows to prove properties of programs (e.g. correctness) in a much more easy way

Operational semantics

We will introduce the operational semantics of logic programming by defining a *transition system* which describe how the interpreter of the language works. The operational semantics corresponds to resolution, but provides a more clear definition of the computation, and can be understood also without considering logic at all.

Declarative languages

Logic programming languages are *declarative* languages, in the sense the the programmer specifies the "what", while the machine takes care of the "how".

Preliminaries, syntax

We will provide the **Syntax** of logic programming languages by using EBNF defined as follows.

- The *extended Backus-Naur form* (EBNF) is a notation consisting of production rules of the form:

Name: $G, H ::= A \mid B, \text{Condition}$

where

- ▶ *capital letters* are syntactical entities
- ▶ *symbols G and H* are defined by the rule with name *Name*.
- ▶ if *Condition* holds then *G* and *H* can be of the form *A* or *B*.

Preliminaries, semantics

We will provide the **Semantics** of logic programming languages by using State transition systems (also called refined calculus)

- State transition systems provide the **operational** (also called procedural) semantics: this defines how the computation evolves.
- As previously mentioned computation is deduction (resolution): we will use state transition systems to describe resolution steps
- **Note that** operational semantics, as defined by a state transition system, can be understood also **without** any reference to logics!
- A declarative semantics is also possible, of course, but we will not consider it.

(Simple) State Transition System

- A Simple State Transition System (in the following Transition System or TS for brevity) is a pair $(S, \mapsto)$ where
 - ▶ S set of states
 - ▶ $\mapsto$ binary relation over states: *transition relation*
 - ▶ *(state) transition* from S_1 to S_2 possible if (some given) condition holds, i.e. S_1 and S_2 are in relation $\mapsto$, written $S_1 \mapsto S_2$
- distinguished subsets of S : *initial* and *final states*
- $S_1 \mapsto S_2 \mapsto \dots \mapsto S_n$ *derivation (computation)*
- $\mapsto^*$ reflexive-transitive closure of $\mapsto$

Reduction is synonym for transition

Transition Rules

- The relation $\mapsto$ is defined by using transition rules of the form

IF	<i>Condition</i>
THEN	$S \mapsto S'$.

- *(state) transition (reduction, derivation step, computation)* from S to S' possible if the condition *Condition* holds
- straightforward to concretize a calculus (" $\vdash$ ") into a state transition system (" $\mapsto$ ")

Refined Calculus

A **Refined Calculus** is provided by a Triple $(\Sigma, \equiv, T)$ where:

- Σ is a signature for a first-order logic language
- $\equiv$ is a *congruence* (equivalence relation) on states
- $T = (S, \mapsto)$ is a transition system, where a state $s \in S$ represents a logical expression over the signature Σ

Intuitively a refined calculus is a first order language, provided with a congruence (which is used to factorize equivalent formulas and thus reduce the number of rules, see next slides) and a transition system which defined the calculus rules (i.e. the operational semantics).

Congruence

- A congruence is used to identify states which are considered *equivalent* for purpose of computation
- *congruence* instead of modeling with additional transition rules
- formally congruence is an **equivalence relation compatible with the structure**:

(Reflexivity) $A \equiv A$

(Symmetry) If $A \equiv B$ then $B \equiv A$

(Transitivity) If $A \equiv B$ and $B \equiv C$ then $A \equiv C$

Example:

$$(X = 3) \equiv (3 = X)$$

Congruence

<i>Commutativity:</i>	$G_1 \wedge G_2$	$\equiv$	$G_2 \wedge G_1$
<i>Associativity:</i>	$G_1 \wedge (G_2 \wedge G_3)$	$\equiv$	$(G_1 \wedge G_2) \wedge G_3$
<i>Identity:</i>	$G \wedge \top$	$\equiv$	G
<i>Absorption:</i>	$G \wedge \perp$	$\equiv$	$\perp$

derived from tautologies

Example. Transition System for Resolution Calculus 1

The **States** of a Transition System for Resolution Calculus are defined as follows

- A *state* is set of clauses
- *emphinitial* state is a set of clauses representing the theory and the negated consequence
- The *final state* contains the empty clause

Example. Transition System for Resolution Calculus 2

The **transition rules** for propositional logic are the following:

- **Resolvent**

IF $R \vee A \in S$ and $R' \vee \neg A \in S$ for some atom A
THEN $S \mapsto S \cup \{F\}$ where $F = R \vee R'$ is called Resolvent.

- **Factor**

IF $R \vee L \vee L \in S$ for some literal L
THEN $S \mapsto S \cup \{F\}$ where $F = R \vee L$ is called Factor

Example. Transition System for Resolution Calculus 3

The **transition rules** for FOL are the following

- **Resolvent**

IF $R \vee A \in S$ and $R' \vee \neg A \in S$ and
 σ is a most general unifier for the atoms A and A' ,
THEN $S \mapsto S \cup \{F\}$ where $F = (R \vee R')\sigma$ is called Resolvent.

- **Factor**

IF $R \vee L \vee L' \in S$ and
 σ is a most general unifier for the literals L and L' ,
THEN $S \mapsto S \cup \{F\}$ where $F = (R \vee L)\sigma$ is called
Factor